Amauri Hopewell

Southern New Hampshire University

CS-499 Computer Science Capstone

September 20, 2026

Milestone Two: Software Design and Engineering Enhancement

1. What is the artifact, and when was it created?

This is the CS-320 Contact, Task, and Appointment work from summer 2025 (CS-320: Software Testing, Automation, and QA). The original files are in the original/ folder of this zip and at https://github.com/AmauriHopewell/CS320. For this milestone I enhanced Contact.java, which stores an ID, first name, last name, phone, and address. The ID is final, and the constructor has a series of safeguards including rejecting nulls, names longer than 10 characters, addresses longer than 30, and phones that are not ten digits. ContactService still adds, updates, deletes, and looks up by ID. I wrote those rules in CS-320 with JUnit. This week's work is the layer around that service, not a rewrite of the entity.

2. Why this artifact, and how was it improved?

I picked Contact because it already had the field rules and tests, and it was still a prototype which gave room for improvements. For instance, state died when the process ended, there was no HTTP API, and there was no login, even though the records are personal. The README also said it used HashMap while ContactService walked an ArrayList. The Module One plan was to keep ContactService as the business layer and put a repository, SQLite, a small REST controller, an auth filter, and an audit log around it. To improve it, I implemented that plan. Now, ContactRepository is the new storage. InMemoryContactRepository is a HashMap. Unit tests use that, so lookup is O(1) and the original Contact / ContactService cases still run. I have also made

a production store in SqliteContactRepository. To test that, a test writes a contact, opens a new service on the same file, and reads it back. I also placed ContactApi as a JDK HttpServer

I have also improved AuthService. AuthService has two users, staff and admin, with passwords encrypted in salted SHA-256, taken from CONTACT_STAFF_PASSWORD and CONTACT_ADMIN_PASSWORD. App.main exits if those are missing, so there is no default password in source. A missing token returns 401 with {}. A staff token on POST, PUT, or DELETE returns 403 with {} and no contact body. AuditLog records actor, action, id, and status. It does not take a phone number or address as arguments. I also named the 10 / 30 limits as constants and removed the System.out.println calls from getContact. To ensure the upgrades work, I added 37 tests using Maven.

3. Course outcomes from Module One

This artifact shows outcomes 3, 4, and 5. Outcome 3 is the store trade-off: HashMap for fast tests, SQLite so contacts survive a restart. Outcome 4 is JUnit, a repository, SQLite JDBC, and java.util.logging used on one path. Outcome 5 is the auth filter and the log that skips PII. Outcome 2 is satisfied by this write-up. Finally, outcome 1 is the GitHub history after I take comments and push the enhanced tree. The 3-2 directions said to wait for instructor comments before putting this on the ePortfolio (SNHU, n.d.), so Pages still links to the original CS-320 repo.

4. What I learned, and what got in the way

The awkward part was moving storage without breaking the tests. ContactService used to use an ArrayList, but the upgrade gave it a ContactRepository and a no-arg constructor that still builds the in-memory map. The old tests construct new ContactService() and do not know about SQLite, while the SQLite tests use a temp file from JUnit.

Also, it made me learn how it can be difficult to create a secure login function. Auth was picky in a useful way. It is easy to return 403 with a JSON error that still includes the contact, potentially exposing it, so I wrote a test that fails if that body contains the sample phone number. For security, Login uses POST /login, while Tokens are random hex, not the password.

Acknowledgment of generative AI use

I used Grok 4.6 (xAI) to help organize this narrative against the Milestone Two prompts and to format the Word file, and to write some of the upgraded code (including all of the unit tests).